

SUBMISSION FORMAT

Sample Repo — What I Expect

```
aws-lambda-boto3-assignments/
├── README.md (index: name, batch, links)
├── 01-s3-bucket-cleanup/
│   ├── lambda_function.py
│   ├── iam_policy.json (proves least-privilege)
│   ├── README.md (8-section docs)
│   └── screenshots/
│       ├── 01-iam-role.png
│       ├── 02-lambda-config.png
│       ├── 03-test-invocation.png
│       ├── 04-cloudwatch-logs.png
│       └── 05-final-result.png
├── 02-ec2-snapshot-cleanup/ ...
└── 04-public-bucket-audit/ ...
```

Repo rules

- ONE repo, one folder per task — submit a single GitHub link
- Screenshots numbered in the order the work happened — I follow your assignment from filenames alone
- iam_policy.json = the exact policy you attached
- README follows the 8-section template from the demo
- **NEVER commit AWS credentials. Account IDs in ARNs are fine; access keys are not.**

Bonus: this repo doubles as an interview portfolio — previous batches have shown it in interviews.

Description

AWS Automation with Lambda & Boto3

Submission Requirements (applies to all assignments):

- Push the Python code to GitHub and share the repository link.
- Provide a brief document explaining the steps you followed.
- Include screenshots of:
 - IAM Role
 - Lambda Configuration
 - Test Invocation/Output
 - CloudWatch Logs
 - Final Result

Your submission will be evaluated on:

- Correctness of the solution
- Successful end-to-end implementation
- Clarity and organization of the documentation
- Assessment Weightage: This assignment carries 50 marks and is an important component of your Final Certification. Ensure your solution is complete, well-documented, and thoroughly tested before submission.

Note: You only need to complete any 4 assignments for grading. However, practicing all the assignments is highly recommended, as these automation patterns are commonly used in real-world DevOps roles and technical interviews.

General notes for all assignments:

- Use Python 3.12 or later as the Lambda runtime (Python 3.7/3.8 runtimes are deprecated).
- Use Amazon EventBridge for scheduling and event rules (the service formerly called CloudWatch Events).
- Use t3.micro for any EC2 instances (free-tier eligible in most regions).
- For IAM, write least-privilege inline policies wherever possible instead of attaching *FullAccess managed policies. This is what interviewers and real environments expect.
- Always print/log affected resource IDs so results are visible in CloudWatch Logs.

Important : Cost Management Tips

If you clean up your resources after testing, these assignments should cost little to nothing.

Keep these points in mind:

- Cost Explorer API (Assignment 4): Approximately ₹1 per API call. Test only a few times and do not schedule it.
- EC2: Stop or terminate instances immediately after capturing your screenshots.
- Snapshots & S3 Buckets: Delete them once your testing is complete.

- Elastic IPs: Release any unused Elastic IPs.
- Best Practices
- Use one AWS Region throughout (recommended: us-east-1).
 - Complete the assignment and clean up resources in the same sitting.
 - Set up a \$1 AWS Budget Alert before you begin.
 - Before logging out, ensure there are no unused EC2 instances, S3 buckets, or snapshots.
- ⚠ Most common mistake: Forgetting to terminate an EC2 instance, which can lead to unexpected AWS charges.

1. Automated S3 Bucket Cleanup (Objects Older Than 30 Days)

Objective: Automate deletion of stale objects in an S3 bucket.

Task: Delete files older than 30 days in a specific bucket.

Instructions:

1. S3 Setup: Create a bucket and upload several files. (Since you can't easily create "old" objects, temporarily lower the age threshold to minutes for testing — then set it back to 30 days in the final code.)
2. Lambda IAM Role: Inline policy with s3:ListBucket and s3:DeleteObject scoped to your bucket.
3. Lambda Function (Python 3.12+, Boto3):
4. List objects in the bucket (use the paginator — never assume one page of results).
5. Compare each object's LastModified (timezone-aware) with the current UTC time.
6. Delete objects older than 30 days.
7. Print the names of deleted objects.
8. Testing: Manually trigger and confirm only newer files remain.
9. Discussion point (include in your documentation): In production, S3 Lifecycle Rules handle this natively with zero code. Explain in 2–3 lines when you'd use Lambda instead (e.g., conditional logic, naming patterns, cross-service actions).

2. Automated EBS Snapshot Creation and Cleanup

Objective: Automate EBS volume backups and delete snapshots older than a retention period.

Instructions:

10. EBS Setup: Identify or create an EBS volume; note the volume ID.
11. Lambda IAM Role: Inline policy with ec2:CreateSnapshot, ec2:DescribeSnapshots, ec2:DeleteSnapshot, ec2:CreateTags.
12. Lambda Function (Boto3):
13. Create a snapshot of the specified volume and tag it (e.g., CreatedBy=Lambda-Backup).
14. List snapshots with that tag (owned by your account) and delete those older than 30 days.
15. Print IDs of created and deleted snapshots.
16. EventBridge: Schedule the function weekly.
17. Testing: Trigger manually; confirm snapshot creation and cleanup in the EC2 console.
18. Discussion point: AWS Data Lifecycle Manager (DLM) does this natively. Note in your documentation when Lambda is still the better choice (custom retention logic, cross-account copies, notifications).

3. Auto-Tagging EC2 Instances on Launch

Objective: Automatically tag newly launched EC2 instances for resource tracking, ownership, and cost allocation.

Instructions:

19. Lambda IAM Role: Inline policy with ec2:CreateTags and ec2:DescribeInstances.
20. Lambda Function (Boto3):
21. Extract the instance ID from the EventBridge event (detail.instance-id).
22. Tag the instance with LaunchDate=<current date> and a custom tag (e.g., Owner or Environment).
23. Print a confirmation message.
24. EventBridge Rule: Create a rule matching event pattern — source aws.ec2, detail-type EC2 Instance State-change Notification, state running — with the Lambda as target.
25. Testing: Launch a new instance; after a short delay, confirm the tags appear.
26. Bonus: Extract the launching IAM user from CloudTrail events and add an Owner tag automatically — this is a popular interview scenario.

4. Daily AWS Cost Alert Using Cost Explorer API and SNS

Objective: Build an automated alert when AWS spend exceeds a threshold.

Note: The old CloudWatch "Billing" metric is legacy — it only exists in us-east-1 and must be manually enabled. The modern, interview-relevant approach uses the Cost Explorer API (ce:GetCostAndUsage).

Instructions:

27. SNS Setup: Create a topic and subscribe your email (confirm the subscription email).
28. Lambda IAM Role: Inline policy with ce:GetCostAndUsage and sns:Publish (scoped to your topic).
29. Lambda Function (Boto3):
30. Initialize ce and sns clients.
31. Query month-to-date UnblendedCost with get_cost_and_usage.
32. Compare against a threshold (e.g., \$50).
33. If exceeded, publish an SNS alert with the current spend.
34. Print the retrieved amount for logging.
35. EventBridge: Schedule daily.
36. Testing: Trigger manually with a low threshold (e.g., \$0.01) to force an alert.
37. Discussion point: Mention AWS Budgets as the managed alternative and when custom Lambda logic wins (per-service breakdowns, Slack/Teams delivery, anomaly logic).

5.Restore an EC2 Instance from the Latest Snapshot

Objective: Automate disaster-recovery: rebuild an instance from its most recent EBS snapshot.

Instructions:

38. Prerequisite: At least one snapshot of the source instance's root volume exists (Graded 2 pairs well here).
39. Lambda IAM Role: ec2:DescribeSnapshots, ec2:RegisterImage (or ec2:CreateImage), ec2:RunInstances, ec2:DescribeImages, ec2:CreateTags.
40. Lambda Function (Boto3):
41. Find the most recent snapshot for the given volume/instance (sort describe_snapshots by StartTime).
42. Register an AMI from the snapshot with register_image (specify root device mapping).
43. Launch a new t3.micro instance from that AMI and tag it (e.g., RestoredFrom=<snapshot-id>).
44. Print the new instance ID.
45. Testing: Trigger manually; verify the new instance boots and contains the snapshot's data. Terminate test instances afterwards to avoid charges.

6.Audit S3 Buckets for Public Access and Notify

Objective: Detect any bucket that is publicly accessible and alert via SNS.

Note: Since April 2023, new buckets have Block Public Access enabled and ACLs disabled by default — so your audit must check both the Block Public Access configuration and bucket policy status, not just ACLs.

Instructions:

46. SNS Setup: Topic + email subscription.
47. Lambda IAM Role: s3:ListAllMyBuckets, s3:GetBucketPublicAccessBlock, s3:GetBucketPolicyStatus, s3:GetBucketAcl, sns:Publish.
48. Lambda Function (Boto3):
49. List all buckets.
50. For each: check get_public_access_block, get_bucket_policy_status (IsPublic flag), and ACL grants.
51. If any bucket is public (or has Block Public Access disabled), publish an SNS alert naming it.
52. EventBridge: Schedule daily.
53. Testing: Deliberately disable Block Public Access on one test bucket and attach a public-read bucket policy; confirm the alert. Re-secure the bucket immediately after testing.

From <<https://vlearnv.herovired.com/page-activity/554/assign/138229>>

Project 1: Automated S3 Bucket Cleanup

Sunday, July 12, 2026 4:55 PM

Task 1: Project 1: Automated S3 Bucket Cleanup (Objects Older Than 30 Days)

Step 1: S3 Bucket Setup

Option -1

AWS Console Navigation

1. Sign in to the **AWS Management Console**.
2. In the top search bar, type **S3** and open the **S3** service.
3. Click **Create bucket**.
4. **Bucket name:** my-cleanup-demo-bucket-12345 (must be globally unique — add your own suffix, e.g. initials or numbers).
5. **AWS Region:** choose your working region (e.g., us-east-1).
6. Leave **Block all public access** checked (default — keep it on).
7. Leave other settings default, scroll down, click **Create bucket**.
8. Open the bucket → click **Upload** → **Add files** → select 3–4 test files (e.g., file1.txt, file2.txt, file3.txt) → click **Upload**.

Amazon S3 > Buckets > Create bucket

Create bucket [info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [info](#)

my-cleanup-s3-demo-bucket

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ ACLs enabled

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☒ Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access/permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☒ Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☒ Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Option -2

CLI Equivalent:

```
aws s3 mb s3://my-cleanup-demo-bucket --region us-east-1
```

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 mb s3://my-cleanup-demo-bucket --region us-east-1
make_bucket: my-cleanup-demo-bucket
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls
2026-07-13 13:51:19 my-cleanup-demo-bucket
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Create test files:

```
echo "test file 1" > file1.txt
```

```
echo "test file 2" > file2.txt
```

```
echo "test file 3" > file3.txt
```

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 1" > file1.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 2" > file2.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 3" > file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls -ltr
Get-ChildItem: A parameter cannot be found that matches parameter name 'ltr'.
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls

Directory: D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup

Mode                LastWriteTime         Length Name
----                -
-a---             7/13/2026  1:56 PM             13 file1.txt
-a---             7/13/2026  1:56 PM             13 file2.txt
-a---             7/13/2026  1:56 PM             13 file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Upload

```
aws s3 cp file1.txt s3://my-cleanup-demo-bucket/
```

```
aws s3 cp file2.txt s3://my-cleanup-demo-bucket/
```

```
aws s3 cp file3.txt s3://my-cleanup-demo-bucket/
```

```

PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file1.txt s3://my-cleanup-demo-bucket/
upload: .\file1.txt to s3://my-cleanup-demo-bucket/file1.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file2.txt s3://my-cleanup-demo-bucket/
upload: .\file2.txt to s3://my-cleanup-demo-bucket/file2.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file3.txt s3://my-cleanup-demo-bucket/
upload: .\file3.txt to s3://my-cleanup-demo-bucket/file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>

```

Verify

aws s3 ls s3://my-cleanup-demo-bucket/

```

PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 13:58:06      13 file1.txt
2026-07-13 13:58:16      13 file2.txt
2026-07-13 13:58:23      13 file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>

```

#Verify from UI:

The screenshot shows the Amazon S3 console interface. On the left, the navigation pane is open, showing 'Amazon S3' > 'Buckets'. The main content area is titled 'Buckets' and has tabs for 'General purpose buckets' (selected) and 'Directory buckets'. Under 'General purpose buckets', there is a sub-header 'General purpose buckets (1)' with an 'Info' link. Below this, there is a search bar 'Find buckets by name' and a table with columns: Name, AWS Region, and Creation date. The table contains one entry: 'my-cleanup-demo-bucket' in the 'US East (N. Virginia) us-east-1' region, created on 'July 13, 2026, 13:51:19 (UTC+05:30)'. Above the table, there are buttons for 'Copy ARN', 'Empty', 'Delete', and 'Create bucket'.

The screenshot shows the Amazon S3 console interface for the 'my-cleanup-demo-bucket'. The left navigation pane is open, showing 'Amazon S3' > 'Buckets' > 'my-cleanup-demo-bucket'. The main content area is titled 'my-cleanup-demo-bucket' with an 'Info' link. Below the title, there are tabs for 'Objects' (selected), 'Metadata', 'Properties', 'Permissions', 'Metrics', 'Management', 'File systems - new', and 'Access Points'. Under the 'Objects' tab, there is a sub-header 'Objects (3)' with an 'Info' link. Below this, there is a search bar 'Find objects by prefix' and a table with columns: Name, Type, Last modified, Size, and Storage class. The table contains three entries: 'file1.txt', 'file2.txt', and 'file3.txt', all of type 'txt', created on 'July 13, 2026, 13:58:06 (UTC+05:30)', 'July 13, 2026, 13:58:16 (UTC+05:30)', and 'July 13, 2026, 13:58:23 (UTC+05:30)' respectively, with a size of '13.0 B' and storage class of 'Standard'. Above the table, there are buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'.

Note on testing "old" files: S3 sets Last Modified automatically at upload time — you cannot backdate it. So for testing, temporarily lower the age threshold to a few minutes (via a Lambda environment variable), upload files, wait past that window, run the function, and confirm deletion. Then switch the threshold back to 30 days for the "production" version.

Step 2: Lambda IAM Role

AWS Console Navigation

1. Search bar → type **IAM** → open **IAM** service.
2. Left sidebar → **Roles** → **Create role**.
3. **Trusted entity type**: AWS service.
4. **Use case**: select **Lambda** → click **Next**.
5. On the **Add permissions** page, skip attaching a managed policy for now (we'll add an inline policy after creation) → click **Next**.
6. **Role name**: s3-cleanup-lambda-role → click **Create role**.
7. Open the newly created role → **Add permissions** dropdown → **Create inline policy**.
8. Switch to the **JSON** tab and paste the policy below (see JSON block).
9. Click **Next**, name it s3-cleanup-inline-policy, click **Create policy**.
10. Also attach the AWS managed policy **AWSLambdaBasicExecutionRole** (Add permissions → Attach policies → search and select it) so the function can write to CloudWatch Logs.

Policy was successfully attached to role.

s3-cleanup-lambda-role

Allows Lambda functions to call AWS services on your behalf.

Summary

Creation date: July 13, 2026, 14:14 (UTC+05:30)

ARN: `arn:aws:iam::675134942081:role/s3-cleanup-lambda-role`

Last activity: -

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2)

You can attach up to 10 managed policies.

Policy name	Type	Attached entities
AWSLambdaBasicExecutionRole	AWS managed	2
s3-cleanup-inline-policy	Customer inline	0

Permissions boundary (not set)

s3-cleanup-inline-policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::my-cleanup-demo-bucket"
    },
    {
      "Effect": "Allow",
      "Action": "s3:DeleteObject",
```

```

        "Resource": "arn:aws:s3:::my-cleanup-demo-bucket/*"
    },
    {
        "Effect": "Allow",
        "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents"
        ],
        "Resource": "arn:aws:logs:*:*:*"
    }
]
}

```

☐ [s3-cleanup-inline-policy](#)
Customer inline
0

s3-cleanup-inline-policy

Copy JSON

Edit ↗

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "s3:ListBucket",
7       "Resource": "arn:aws:s3:::my-cleanup-demo-bucket-12345"
8     },
9     {
10      "Effect": "Allow",
11      "Action": "s3:DeleteObject",
12      "Resource": "arn:aws:s3:::my-cleanup-demo-bucket-12345/*"
13    },
14    {
15      "Effect": "Allow",
16      "Action": [
17        "logs:CreateLogGroup",
18        "logs:CreateLogStream",
19        "logs:PutLogEvents"
20      ],

```

▶ Permissions boundary (not set)

AWSLambdaBasicExecutionRole:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}

```


AWSLambdaBasicExecutionRole

Provides write permissions to CloudWatch Logs.

Copy JSON

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "logs:CreateLogGroup",  
8         "logs:CreateLogStream",  
9         "logs:PutLogEvents"  
10      ],  
11      "Resource": "*"   
12    }  
13  ]  
14 }
```

Important: `s3:ListBucket` applies to the **bucket ARN** (no trailing `/*`), while `s3:DeleteObject` applies to **object ARNs** (with `/*`). Mixing these up is the most common cause of `AccessDenied` errors.

Step 3: Lambda Function (Python 3.12, Boto3)

AWS Console Navigation

1. Search bar → type **Lambda** → open **Lambda** service.
2. Click **Create function**.
3. Choose **Author from scratch**.
4. **Function name:** `s3-stale-object-cleanup`.
5. **Runtime:** Python 3.12.
6. **Architecture:** `x86_64` (default is fine).
7. Expand **Change default execution role** → select **Use an existing role** → choose `s3-cleanup-lambda-role`.
8. Click **Create function**.
9. In the **Code** tab, delete the placeholder code in `lambda_function.py` and paste the code below.
10. Click **Deploy** (top left of the code editor) to save.
11. Go to the **Configuration** tab → **Environment variables** → **Edit** → **Add environment variable**:
 - Key: `BUCKET_NAME`, Value: `my-cleanup-demo-bucket-12345`
 - Key: `AGE_THRESHOLD_DAYS`, Value: `30`
 - Save.
12. Still under **Configuration** → **General configuration** → **Edit** → set **Timeout** to 1 min 0 sec (default 3 sec is too short) → **Save**.

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
 Start with a simple Hello World example.

☐ **Use a blueprint**
 Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
 Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

s3-stale-object-cleanup

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.12

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution** role under **Additional settings**.

Custom settings [Info](#)

Durable execution - new [?](#)
 Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - new [?](#)
 Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

Additional settings

Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General [Info](#)

ARM64 architecture
 Use ARM64 instead of x86_64 (default).

Custom execution role
 Use custom execution role instead of new role with basic Lambda permissions (default).

Networking [Info](#)

Function URL
 Assign HTTP(S) endpoints.

Tenant isolation mode - new [?](#)
 Guarantee separate execution environments for each tenant invoking the function.

VPC
 Connect to a VPC to securely access private resources.

Configure custom execution role

Choose an existing execution role or create a new one.

Execution role

To access other AWS services and resources your function needs an IAM role.

Choose a role with the right permissions for your function.

s3-cleanup-lambda-role

[Create new role](#)

[Edit role](#)

[View role details in IAM](#)

Close

Save

Custom settings
Info

Durable execution - new
Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General
Info

ARM64 architecture
Use ARM64 instead of x86_64 (default).

Custom execution role
Use custom execution role instead of new role with basic Lambda permissions (default).
IAM permissions
s3-cleanup-lambda-role

lambda_function.py:

```
import boto3
import os
from datetime import datetime, timezone, timedelta

s3 = boto3.client('s3')

BUCKET_NAME = os.environ['BUCKET_NAME']
#AGE_THRESHOLD_DAYS = int(os.environ.get('AGE_THRESHOLD_DAYS', '1'))
# Optional override for quick testing (e.g. "2" = 2 minutes)
AGE_THRESHOLD_MINUTES = os.environ.get('AGE_THRESHOLD_MINUTES')

def lambda_handler(event, context):
    if AGE_THRESHOLD_MINUTES:
        cutoff = datetime.now(timezone.utc) - timedelta(minutes=int(AGE_THRESHOLD_MINUTES))
    else:
        cutoff = datetime.now(timezone.utc) - timedelta(days=AGE_THRESHOLD_DAYS)

    paginator = s3.get_paginator('list_objects_v2')
    page_iterator = paginator.paginate(Bucket=BUCKET_NAME)

    deleted = []
    scanned = 0

    for page in page_iterator:
        contents = page.get('Contents', [])
        objects_to_delete = []

        for obj in contents:
            scanned += 1
            key = obj['Key']
            last_modified = obj['LastModified'] # timezone-aware, UTC
```

```

if last_modified < cutoff:
    objects_to_delete.append({'Key': key})

# Batch delete (S3 allows up to 1000 keys per call)
if objects_to_delete:
    response = s3.delete_objects(
        Bucket=BUCKET_NAME,
        Delete={'Objects': objects_to_delete, 'Quiet': True}
    )
    deleted.extend([o['Key'] for o in objects_to_delete])

errors = response.get('Errors', [])
if errors:
    print(f"Errors during delete: {errors}")

print(f"Scanned {scanned} objects. Deleted {len(deleted)} objects.")
for key in deleted:
    print(f"Deleted: {key}")

return {
    'statusCode': 200,
    'scanned': scanned,
    'deleted_count': len(deleted),
    'deleted_keys': deleted
}

```

The screenshot shows the AWS Lambda console interface for the function 's3-stale-object-cleanup'. The 'Code source' tab is selected, showing the Python code for the lambda function. The code includes imports for boto3, os, and datetime, and defines a lambda_handler function that scans for stale objects and deletes them. The 'Deploy' button is highlighted with a red box. The right sidebar shows the 'Info' and 'Tutorial' tabs, with the 'Tutorial' tab selected, displaying 'Choose a tutorial' and 'Create a simple' sections.

Edit environment variables

Environment variables

You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. [Learn more](#)

Key	Value	
BUCKET_NAME	my-cleanup-demo-bucket	<button>Remove</button>
AGE_THRESHOLD_DAYS	1	<button>Remove</button>

Add environment variable

► Encryption configuration

[Cancel](#)

[Save](#)

Updating the function "s3-stale-object-cleanup".

[Diagram](#) [Template](#)



[+ Add trigger](#)

[+ Add destination](#)

Function ARN

arn:aws:lambda:us-east-1:675134942081:function:s3-stale-object-cleanup

Description

-

Last modified

4 minutes ago

[Code](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)

Configuration

[General configuration](#)

[Triggers](#)

[Permissions](#)

[Destinations](#)

[Function URL](#)

[Environment variables](#)

Environment variables (2)

[Edit](#)

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
AGE_THRESHOLD_DAYS	1
BUCKET_NAME	my-cleanup-demo-bucket

s3-stale-object-cleanup Throttle Copy ARN Actions

▼ **Function overview** Info

Diagram | Template

 **s3-stale-object-cleanup**

 Layers (0)

+ Add trigger + Add destination

Function ARN
arn:aws:lambda:us-east-1:675134942081:function:s3-stale-object-cleanup

Description
-

Last modified
2 minutes ago

Export to Infrastructure Composer Download

Configuration Info

General configuration Info Edit

Description -	Memory 128 MB	Ephemeral storage 512 MB
Timeout 1 min 0 sec	SnapStart None	

Configuration

- General configuration**
- Triggers
- Permissions
- Destinations

Step 4: Testing

Phase A — Test with minutes threshold

Console:

1. Go to Lambda function → **Configuration** → **Environment variables** → **Edit**.
2. Add a new variable: Key AGE_THRESHOLD_MINUTES, Value 2 → **Save**.
3. Upload a couple of test files to the bucket now (S3 console → Upload).
4. Wait 2–3 minutes.
5. Go to the **Test** tab → **Create new event** → Event name: manualTest → keep the default empty JSON {} (the function doesn't need event input) → **Save** → click **Test**.
6. Check the **Execution results** panel for output, and check **Monitor** tab → **View CloudWatch logs** to see the printed deleted key names.

Before run function:

```
upload: ./files.txt to s3://my-cleanup-demo-bucket/files.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:21:05      13 file1.txt
2026-07-13 15:21:12      13 file2.txt
2026-07-13 15:23:28      13 file3.txt
2026-07-13 15:23:22      13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Amazon S3 > Buckets > my-cleanup-demo-bucket

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

my-cleanup-demo-bucket info

Objects Metadata Properties Permissions Metrics Management File systems - new Access Points

Objects (4)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	file1.txt	txt	July 13, 2026, 15:21:05 (UTC+05:30)	13.0 B	Standard
☐	file2.txt	txt	July 13, 2026, 15:21:12 (UTC+05:30)	13.0 B	Standard
☐	file3.txt	txt	July 13, 2026, 15:23:28 (UTC+05:30)	13.0 B	Standard
☐	file4.txt	txt	July 13, 2026, 15:23:22 (UTC+05:30)	13.0 B	Standard

After manual run Lambda function:

Lambda > Functions > s3-stale-object-cleanup

Summary

Code SHA-256
l+shTF8/m36ku1qaTB+s8ZPWSuiecw8vjC4+KnF39Mw=

Function version
\$LATEST

Duration
688.16 ms

Resources configured
128 MB

Init duration
502.17 ms

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 189d2e04-44a6-4cf7-97c4-f874643f52a3 Version: $LATEST
Scanned 4 objects. Deleted 4 objects.
Deleted: file1.txt
Deleted: file2.txt
Deleted: file3.txt
Deleted: file4.txt
END RequestId: 189d2e04-44a6-4cf7-97c4-f874643f52a3
REPORT RequestId: 189d2e04-44a6-4cf7-97c4-f874643f52a3 Duration: 688.16 ms Billed Duration: 1191 ms Memory Size: 128 MB Max Memory Used: 97 MB Init Duration: 502.17 ms
```

Execution time
2 seconds ago

Request ID
189d2e04-44a6-4cf7-97c4-f874643f52a3

Billed duration
1191 ms

Max memory used
97 MB

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:21:05      13 file1.txt
2026-07-13 15:21:12      13 file2.txt
2026-07-13 15:23:28      13 file3.txt
2026-07-13 15:23:22      13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> []
```

Phase B — Reset to production threshold

Delete the AGE_THRESHOLD_MINUTES variable entirely (leave AGE_THRESHOLD_DAYS=1) → Save.

Successfully updated the function "s3-stale-object-cleanup".

Layers (0)

+ Add trigger

+ Add destination

Description

Last modified 5 minutes ago

Code | Test | Monitor | **Configuration** | Aliases | Versions

Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Environment variables (2) [Edit](#)

The environment variables below are encrypted at rest with the default Lambda service key.

Q Search Environment variables

Key	Value
AGE_THRESHOLD_DAYS	1
BUCKET_NAME	my-cleanup-demo-bucket

ReTest:

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:56:25      13 file1.txt
2026-07-13 15:56:18      13 file2.txt
2026-07-13 15:56:07      13 file3.txt
2026-07-13 15:56:14      13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

After run Lambda function

Lambda > Functions > s3-stale-object-cleanup

Function version: \$LATEST

Duration: 587.66 ms

Resources configured: 128 MB

Init duration: 490.47 ms

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: b3d9ba68-ad7a-4a84-8671-9b4580401cc5 Version: $LATEST
Scanned 4 objects. Deleted 0 objects.
END RequestId: b3d9ba68-ad7a-4a84-8671-9b4580401cc5
REPORT RequestId: b3d9ba68-ad7a-4a84-8671-9b4580401cc5 Duration: 587.66 ms Billed Duration: 3879 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 490.47 ms
```

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:56:25      13 file1.txt
2026-07-13 15:56:18      13 file2.txt
2026-07-13 15:56:07      13 file3.txt
2026-07-13 15:56:14      13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Checking CloudWatch Logs (Console)

1. Search bar → **CloudWatch** → **Log groups**.
2. Find /aws/lambda/s3-stale-object-cleanup.
3. Open the latest **Log stream** to view scanned/deleted counts and the list of deleted keys.

The screenshot shows the AWS CloudWatch console. The breadcrumb navigation at the top indicates the path: CloudWatch > Log management > /aws/lambda/s3-stale-object-cleanup > 2026/07/13/[SLATEST]ac5d314fbc2246e28f73fbb8452ceff. The left sidebar shows the 'Log Management' section expanded. The main area displays 'Log events' for the specified function. A search bar is present with the text 'Filter events - press enter to search'. Below the search bar, there are tabs for 'Timestamp' and 'Message'. The log events are listed in a table with columns for 'Timestamp' and 'Message'. The messages include 'INIT_START Runtime Version: python:3.12...mainlinev2.v14 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:9a45ef23f13ca371f8ee68571c08acfb9a5549dcb2a1409f4e59ba45', 'START RequestId: b3d9ba68-ad7a-4a84-8671-9a4380481cc5 Version: SLATEST', 'Scanned 4 objects. Deleted 0 objects.', 'END RequestId: b3d9ba68-ad7a-4a84-8671-9a4380481cc5', and 'REPORT RequestId: b3d9ba68-ad7a-4a84-8671-9a4380481cc5 Duration: 587.66 ms Billed Duration: 1879 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 498.47 ms'. There are also buttons for 'Actions', 'Start tailing', and 'Create metric filter'.

Step 5 (Optional): Schedule with EventBridge instead of manual trigger

If you want this to run automatically (e.g., daily) rather than being manually invoked:

Console:

1. Go to the Lambda function → **Add trigger**.
2. Select **EventBridge (CloudWatch Events)**.
3. Choose **Create a new rule**.
4. Rule name: `daily-s3-cleanup-schedule`.
5. Rule type: **Schedule expression** → enter `rate(1 day)` (or a cron expression like `cron(0 3 * * ? *)` for 3 AM daily).
6. Click **Add**.

The screenshot shows the AWS Lambda console for the function 's3-stale-object-cleanup'. The breadcrumb navigation at the top indicates the path: Lambda > Functions > s3-stale-object-cleanup. The left sidebar shows the 'Configuration' section expanded. The main area displays the 'Configuration' tab for the function. There are buttons for '+ Add trigger' and '+ Add destination'. The 'Triggers' section is highlighted, and the 'Add trigger' button is circled in red. The 'Triggers' section shows a search bar and a table with columns for 'Trigger'.

Add trigger

Trigger configuration [Info](#)

EventBridge (CloudWatch Events)
aws asynchronous schedule management-tools

Rule
Pick an existing rule, or create a new one.
☒ Create a new rule
☐ Existing rules

Rule name
Enter a name to uniquely identify your rule.
daily-s3-cleanup-schedule

Rule description
Provide an optional description for your rule.

Rule type
Trigger your target based on an event pattern, or based on an automated schedule.
☐ Event pattern
☒ Schedule expression

Schedule expression
Self-trigger your target on an automated schedule using [Cron or rate expressions](#). Cron expressions are in UTC.
rate(1 day),cron(53 10 13 7 *)
e.g. rate(1 day), cron(0 17 ? * MON-FRI *)

Lambda will add the necessary permissions for Amazon EventBridge (CloudWatch Events) to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

[Cancel](#) [Add](#)

EventBridge (CloudWatch Events) [+ Add destination](#) [Last modified 7 minutes ago](#)

[+ Add trigger](#)

Configuration

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases

Triggers (1) [Info](#) [Fix errors](#) [Edit](#) [Delete](#) [Add trigger](#)

Search for triggers

☐ Trigger

EventBridge (CloudWatch Events): daily-s3-cleanup-schedule
arn:aws:events:us-east-1:675134942081:rule/daily-s3-cleanup-schedule
Rule state: **ENABLED**

Details

Event bus: default
IsComplexStatement: No
Schedule expression: rate(10 minutes)
Service principal: events.amazonaws.com
Statement ID: lambda-c55e94b2-8cb4-40c8-b4d9-0606ab0f2955

Amazon EventBridge > **Scheduled rules** > daily-s3-cleanup-schedule

daily-s3-cleanup-schedule [Edit](#) [Disable](#) [Delete](#) [CloudFormation Template](#)

Rule details [Info](#)

Rule name daily-s3-cleanup-schedule	Status ✔ Enabled	Event bus name default	Type Scheduled Standard
Description -	Rule ARN arn:aws:events:us-east-1:675134942081:rule/daily-s3-cleanup-schedule	Event bus ARN arn:aws:events:us-east-1:675134942081:event-bus/default	

Event schedule [Info](#) [Edit](#)

Fixed rate of
10 minute

After 10 min execute :

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

CloudWatch

Log management

/aws/lambda/s3-static-object-cleanup

2026/07/15/\$LATEST31adff6469574c7586bb7094e8fdae32

CloudWatch

Log events

Actions

Start tailing

Create metric filter

Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

Timestamp	Message
No older events at this moment. Retry	
2026-07-17T10:47:02.996Z	INIT_START Runtime Version: python:3.12.x64[linux], v54 Runtime Version ARM: arm:aws:lambda:us-east-1::runtime:ba5ef23f17ace13ca371f8ee68571c88acfb89a5568bdc8d2a1a094e9ba45
2026-07-17T10:47:03.396Z	START RequestId: 82c1d85d-c3be-4569-b346-c14e7ea4e038 Version: \$LATEST
2026-07-17T10:47:04.122Z	Scanned 4 objects. Deleted 4 objects.
2026-07-17T10:47:04.122Z	Deleted: fl1a1.txt
2026-07-17T10:47:04.122Z	Deleted: fl1a2.txt
2026-07-17T10:47:04.122Z	Deleted: fl1a3.txt
2026-07-17T10:47:04.122Z	Deleted: fl1a4.txt
2026-07-17T10:47:04.342Z	END RequestId: 82c1d85d-c3be-4569-b346-c14e7ea4e038
2026-07-17T10:47:04.342Z	REPORT RequestId: 82c1d85d-c3be-4569-b346-c14e7ea4e038 Duration: 632.58 ms Billed Duration: 1343 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 588.85 ms
2026-07-17T10:47:04.342Z	REPORT RequestId: 82c1d85d-c3be-4569-b346-c14e7ea4e038 Duration: 632.58 ms Billed Duration: 1343 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 588.85 ms
No newer events at this moment. Auto retry paused. Resume	